Week 1 — Detailed Tasks with Resources

All members do this one week of work in parallel. The goal: the whole team gets a working setup and a shared understanding of the project. No board work in week 1.

Shared start (everyone, before your task)

1. Fork the repo into the club GitHub account: https://github.com/slvDev/esp32-ai
2. Clone your fork and add the original as upstream.
3. Read these three files in order:
4. https://github.com/slvDev/esp32-ai/blob/main/README.md
5. https://github.com/slvDev/esp32-ai/blob/main/RESULTS.md
6. https://github.com/slvDev/esp32-ai/blob/main/research/tinystories/README.md
7. Read the master plan in this folder (00_MASTER_PLAN.md).
8. Write 5 lines in the shared Drive doc: what you think the project does and where the hard part is.

The rest of this document is your week-1 task. Finish it before the weekly meetup. At the meetup, one member presents the learning topic. See the meetup schedule in the master plan.

R1 — Data setup

Goal: the tokenized TinyStories data exists and works.

Steps: 1. Set up Python with uv. Run `uv sync` in the repo root. 2. Open Google Colab. Mount Google Drive. 3. Run the data prep: `uv run python -m research.tinystories.prepare --vocab 32768` 4. Move the data folder to Drive so week-2 runs reuse it: `data/tinystories/` 5. Open `data/tinystories/vocab-32768/tokenizer.json` and count the tokens.

Deliverable: a note in the tracking sheet saying data is ready, with the folder path and the token count.

Verify: the prepare command finishes without error. The tokenizer shows 32768 tokens.

Learning: neural network basics. Watch episodes 1 to 3 of the 3Blue1Brown series. Write a small MLP in numpy that learns XOR. Run it and watch the loss fall.

Resources: - uv: https://docs.astral.sh/uv/ - Colab: https://colab.research.google.com - 3Blue1Brown neural networks: https://www.youtube.com/playlist?list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi - TinyStories dataset: https://huggingface.co/datasets/roneneldan/TinyStories - prepare.py source: https://github.com/slvDev/esp32-ai/blob/main/research/tinystories/prepare.py

R2 — Tracking sheet

Goal: one sheet holds every published claim and its status.

Steps: 1. Read RESULTS.md again. List every number in it. 2. Build a Google Sheet. Columns: claim, published value, our value, status, owner. 3. Add one row per claim. Status is "pending" for all rows. 4. Share the sheet with the club Drive folder.

Deliverable: the sheet link in the club Drive, with at least 20 claim rows.

Verify: R1 finds the data claim in the sheet without help.

Learning: neural network basics. Watch episodes 4 to 5 of 3Blue1Brown (backpropagation). Write the XOR MLP from R1 with one hidden layer and backprop.

Resources: - RESULTS.md: https://github.com/slvDev/esp32-ai/blob/main/RESULTS.md - 3Blue1Brown neural networks: https://www.youtube.com/playlist?list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi - micrograd (minimal autograd): https://github.com/karpathy/micrograd

R3 — Training pipeline explainer

Goal: the team understands how the 3 arms compare.

Steps: 1. Read research/tinystories/train.py fully. It is short. 2. Write down every hyperparameter: vocab, steps, target-core, seed, batch, sequence length, learning rate. 3. Find how the arms are sized to the same core budget. The concept is core-matching. 4. Write the 1-page explainer "How the 3 arms compare". Include a table: arm, what it tests, core size, table size.

Deliverable: the explainer as a markdown file in the club repo.

Verify: R1 and R2 read it and can explain core-matching back to you.

Learning: gradient descent and loss. Watch 3Blue1Brown episode 2 again. Explain in your own words what a loss curve shows.

Resources: - train.py source: https://github.com/slvDev/esp32-ai/blob/main/research/tinystories/train.py - TinyStories paper: https://arxiv.org/abs/2305.07759 - 3Blue1Brown neural networks: https://www.youtube.com/playlist?list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi

R4 — Analyzer notes

Goal: week 4 analysis commands are ready and understood.

Steps: 1. Read research/tinystories/analyze.py fully. 2. Document what the analyzer refuses and why: missing arms, wrong seed count, mixed cohorts. 3. Write the week-4 commands into a note. One command must be: `uv run python -m research.tinystories.analyze --tag cleandeploy --expect-arms baseline,ple,fatembed --expect-seeds 2` 4. Run the command with an empty runs folder. Record the error message.

Deliverable: the analyzer notes with the error record.

Verify: the analyzer refuses an empty cohort with a clear error.

Learning: the MLP forward pass. Write the numpy forward pass for the XOR MLP. Print the hidden activations.

Resources: - analyze.py source: https://github.com/slvDev/esp32-ai/blob/main/research/tinystories/analyze.py - research README: https://github.com/slvDev/esp32-ai/blob/main/research/tinystories/README.md - 3Blue1Brown neural networks: https://www.youtube.com/playlist?list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi

R5 — Quantization explainer

Goal: the team understands int4 PTQ before we quantize anything.

Steps: 1. Read research/tinystories/quantize_eval.py and src/quantize.py. 2. Write down what these terms mean: group size, fp16 scales, symmetric quantization, PTQ. 3. Write the 1-page PTQ explainer. Include the published table: baseline and ple at 4-bit.

Deliverable: the explainer as a markdown file in the club repo.

Verify: R2 can answer a 5-question quiz from the explainer.

Learning: neural network basics. Watch 3Blue1Brown episode 3. Explain what a weight matrix does.

Resources: - quantize_eval.py: https://github.com/slvDev/esp32-ai/blob/main/research/tinystories/quantize_eval.py - src/quantize.py: https://github.com/slvDev/esp32-ai/blob/main/src/quantize.py - RESULTS.md (quantization section): https://github.com/slvDev/esp32-ai/blob/main/RESULTS.md - 3Blue1Brown neural networks: https://www.youtube.com/playlist?list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi

H1 — Golden tests

Goal: the C runtime matches PyTorch on this machine.

Steps: 1. Set up the Python env with `uv sync`. 2. Build the portable C runtime on host. See runtime/host_verify. 3. Run the host golden tests: tests/ and runtime/host_verify. 4. Record the max abs diff.

Deliverable: the golden test result in the tracking sheet.

Verify: the diff is 1e-5 or below, the published value.

Learning: what a model does at runtime: token in, logits out. Read the first 100 lines of the runtime main loop. Write what each of these does in one line: embeddings, attention, FFN, output head.

Resources: - runtime/: https://github.com/slvDev/esp32-ai/tree/main/runtime - tests/: https://github.com/slvDev/esp32-ai/tree/main/tests - RESULTS.md (golden claim): https://github.com/slvDev/esp32-ai/blob/main/RESULTS.md

H2 — Deploy pipeline doc

Goal: the pipeline from model file to flashed board is documented.

Steps: 1. Read scripts/fetch_model.sh, scripts/deploy.sh, scripts/benchmark_device.py. 2. Trace the SHA-256 pin logic in fetch_model.sh. Note what must match before install. 3. Write the pipeline doc: steps, diagram, failure points.

Deliverable: the pipeline doc as a markdown file in the club repo.

Verify: H4 follows the doc and can name the three stages of deploy.sh.

Learning: neural network basics. Watch 3Blue1Brown episode 1. Write the MLP from the video in numpy.

Resources: - scripts/: https://github.com/slvDev/esp32-ai/tree/main/scripts - 3Blue1Brown neural networks: https://www.youtube.com/playlist?list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi

H3 — Export explainer

Goal: the team understands the model file format and the export guards.

Steps: 1. Read research/tinystories/export.py and sample.py. 2. Write down what these mean: PLE format, model.bin, golden.txt, golden.npz, tied head, tokenizer hash. 3. Write the 1-page export explainer. Answer in it: why does export refuse a wrong tokenizer?

Deliverable: the export explainer as a markdown file in the club repo.

Verify: you can explain the `--allow-unverified-tokenizer` flag from memory.

Learning: tokenization. Read how the tokenizer maps text to ids. Explain why a 32768-token vocab needs a big table.

Resources: - export.py: https://github.com/slvDev/esp32-ai/blob/main/research/tinystories/export.py - sample.py: https://github.com/slvDev/esp32-ai/blob/main/research/tinystories/sample.py - research README: https://github.com/slvDev/esp32-ai/blob/main/research/tinystories/README.md

H4 — Memory layout map

Goal: the team sees where every weight lives on the chip.

Steps: 1. Read the src/ runtime sources. 2. Find where each of these lives: activations, norm weights, the core, the output head, the PLE table. 3. Map them to the tiers: SRAM, PSRAM, flash. 4. Write the 1-page memory-layout map.

Deliverable: the memory-layout map as a markdown file in the club repo.

Verify: you can answer: why does the head sit in PSRAM and the table in flash?

Learning: next-token prediction. Explain in two sentences why the model reads 6 table rows per token.

Resources: - src/: https://github.com/slvDev/esp32-ai/tree/main/src - RESULTS.md (memory numbers): https://github.com/slvDev/esp32-ai/blob/main/RESULTS.md

H5 — Colab runbook

Goal: any member can run a training run on Colab and survive a disconnect.

Steps: 1. Run a 50-step probe on Colab: `uv run python -m research.tinystories.train --arm baseline --vocab 32768 --steps 50 --target-core 559000 --seed 0` 2. Find where train.py saves checkpoints. Move them to Drive. 3. Kill the session. Reopen it. Resume from the checkpoint. 4. Write the Colab runbook: keep-alive, Drive save, resume, quota notes.

Deliverable: RUNBOOK.md in the club repo.

Verify: a killed session resumes from the last checkpoint without restarting from step 0.

Learning: neural network basics. Watch 3Blue1Brown episode 2. Explain what an epoch is.

Resources: - Colab: https://colab.research.google.com - Kaggle (fallback): https://www.kaggle.com - train.py: https://github.com/slvDev/esp32-ai/blob/main/research/tinystories/train.py - 3Blue1Brown neural networks: https://www.youtube.com/playlist?list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi

E1 — Toolchain and firmware study

Goal: the toolchain works and the firmware sources are mapped.

Steps: 1. Install the ESP32 toolchain. Use Arduino ESP32 core 3.3.10 or ESP-IDF. - Arduino ESP32 core: https://github.com/espressif/arduino-esp32 - ESP-IDF: https://github.com/espressif/esp-idf 2. Build the portable C runtime on host. It must compile without a board. 3. Read the firmware folders: esp32_barista, esp32_tinystories, benchmarks. 4. Write the firmware walkthrough notes: what each folder does.

Deliverable: the walkthrough notes in the club repo.

Verify: the host runtime builds without error. You can name the parts of a firmware sketch.

Learning: the ESP32-S3 memory map. Draw it: 512 KB SRAM, 8 MB PSRAM, 16 MB flash. Label what the model puts in each tier.

Resources: - ESP32-S3 docs: https://docs.espressif.com/projects/esp-idf/en/latest/esp32s3/ - firmware/: https://github.com/slvDev/esp32-ai/tree/main/firmware

E2 — Measurement protocol

Goal: week 5 measurements are defined before the board arrives.

Steps: 1. Build the bandwidth benchmark firmware on host. Compile only, no flash. See firmware/benchmarks/bandwidth. 2. Read the benchmark source. Write down what it measures and how. 3. Write the measurement protocol: tok/s, ms/token, memory free, flash row read, PSRAM and SRAM speeds. For each: the steps, the tool, the recording format, 3 repetitions.

Deliverable: MEASUREMENT_PROTOCOL.md in the club repo.

Verify: E1 reviews the protocol and finds no missing step.

Learning: memory hierarchy. Explain why flash reads are slow and SRAM reads are fast. One sentence each.

Resources: - firmware/benchmarks/bandwidth: https://github.com/slvDev/esp32-ai/tree/main/firmware/benchmarks/bandwidth - RESULTS.md (bandwidth table): https://github.com/slvDev/esp32-ai/blob/main/RESULTS.md - ESP32-S3 docs: https://docs.espressif.com/projects/esp-idf/en/latest/esp32s3/

Week 1 learning topic (meetup)

One member presents "what is a neural network" for 15 minutes. Use the 3Blue1Brown series as the base. The presenter is decided at the first meetup.

Week 1 success criteria

The week is done when:

- Data is prepared and cached in Drive. (R1)

- The tracking sheet holds every claim. (R2)
- Three explainer docs exist in the club repo: arms, PTQ, export. (R3, R5, H3)
- The analyzer notes and the runbook exist. (R4, H5)
- The golden diff is 1e-5 or below. (H1)
- The pipeline doc and the memory map exist. (H2, H4)
- The toolchain builds and the protocol doc exists. (E1, E2)
- Every member can say what the project does in one sentence.